Communication Infrastructure

How I got it running on a fresh Windows computer

Overview: I ran the ProjectN communication infrastructure on Windows by using WSL2 with Ubuntu 22.04, then installing ROS 2 Humble inside Ubuntu. I used this setup because the communication infrastructure is a ROS 2 workspace and it relies on Linux-style ROS commands, bash setup files, and Linux paths. Running it directly on native Windows would likely require extra changes, so WSL2 was the safest and cleanest setup.

Host system	Windows with WSL2
Linux distribution	Ubuntu 22.04
ROS version	ROS 2 Humble
Workspace to build/run	~/projectn/ROS2Coordination/communication_infra

1. I enabled virtualization in BIOS

First, I checked that virtualization was enabled on the computer. I restarted the computer and entered BIOS/UEFI. In BIOS, I enabled Intel Virtualization Technology, Intel VMX, or Intel VT-x. On AMD machines, the equivalent setting is usually SVM Mode or AMD-V.

I had to do this because WSL2 runs Ubuntu inside a virtual machine. Without virtualization enabled, Ubuntu cannot start properly under WSL2. That is after many tires and conclusions after consulting Chatgpt

After saving and restarting Windows, I checked Task Manager > Performance > CPU and confirmed that it showed Virtualization: Enabled.

2. I enabled WSL2 on Windows

Then I opened PowerShell as Administrator and enabled the required Windows features.

```
wsl.exe --install --no-distribution
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
bcdedit /set hypervisorlaunchtype Auto
```

After that, I restarted the computer. I did this because WSL2 needs both the Windows Subsystem for Linux feature and the Virtual Machine Platform feature. Without them, Ubuntu may download but fail to start.

3. I installed Ubuntu 22.04 through PowerShell

After restart, I opened PowerShell as Administrator again.

```
wsl --update
wsl --set-default-version 2
wsl --install --web-download -d Ubuntu-22.04
```

I used Ubuntu 22.04 because the project uses ROS 2 Humble, and ROS 2 Humble is intended for Ubuntu 22.04.

If Ubuntu was partially installed before, I removed it first and then installed it again:

```
wsl --unregister Ubuntu-22.04
wsl --install --web-download -d Ubuntu-22.04
```

Then I checked that Ubuntu was installed:

```
wsl -l -v
```

Expected result:

```
Ubuntu-22.04    Stopped    2
```

Then I started Ubuntu:

```
wsl -d Ubuntu-22.04
```

On the first start, Ubuntu asked me to create a Linux username and password.

4. I updated Ubuntu and installed basic tools

Inside Ubuntu, I first updated the package list and upgraded the system:

```
sudo apt update
sudo apt upgrade -y
```

Then I installed basic tools needed for downloading, building, and extracting the project:

```
sudo apt install -y curl git unzip build-essential python3-pip software-properties-common gnupg
lsb-release nano
```

5. I installed ROS 2 Humble

Still inside Ubuntu, I configured the locale:

```
sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

Then I enabled the Ubuntu universe repository:

```
sudo add-apt-repository universe -y
sudo apt update
```

Then I added the ROS 2 repository key:

```
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg
```

Then I added the ROS 2 apt source:

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-
keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME)
main" \
  | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

Then I updated apt again and installed ROS 2 Humble with the build tools:

```
sudo apt update
sudo apt install -y \
  ros-humble-ros-base \
  ros-dev-tools \
  python3-colcon-common-extensions \
  python3-rosdep \
  python3-yaml
```

Then I initialized rosdep and tested that ROS 2 worked:

```
sudo rosdep init || true
rosdep update
source /opt/ros/humble/setup.bash
ros2 --help
```

If the help text appears, ROS 2 is installed correctly. I installed ros-humble-ros-base because this project is mainly communication infrastructure and does not need the full desktop ROS installation.

6. I copied the ROS2Coordination.zip repo into Ubuntu

The ZIP file was on my Windows Desktop. Inside Ubuntu, I created a project folder:

```
cd ~
mkdir -p ~/projectn
```

Then I copied the ZIP from Windows into Ubuntu. The Windows files are available in WSL through /mnt/c. For example, my Windows username was Othma, so I used:

```
cp /mnt/c/Users/Othma/Desktop/ROS2Coordination.zip ~/projectn/
```

If the username is different, I can check available Windows users with:

```
ls /mnt/c/Users
```

Then I unzipped the project:

```
cd ~/projectn
unzip ROS2Coordination.zip
```

I did this because building directly inside the Windows filesystem can sometimes cause path or permission issues. It is cleaner to copy the repo into the Linux home directory and build it there.

7. I went to the correct ROS 2 workspace

The important folder is:

```
~/projectn/ROS2Coordination/communication_infra
```

So I ran:

```
cd ~/projectn/ROS2Coordination/communication_infra
ls
```

I expected to see:

```
src
AgentHub
How to run 1.0.md
```

This is the actual ROS 2 workspace for the communication infrastructure. I did not build from the repo root and I did not build from the src folder. The correct build location is the communication_infra folder because that folder contains the src folder used by colcon.

8. I built the communication infrastructure

From the communication infrastructure folder, I ran:

```
cd ~/projectn/ROS2Coordination/communication_infra
source /opt/ros/humble/setup.bash
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
```

I sourced /opt/ros/humble/setup.bash first because that loads the ROS 2 Humble environment. Then I used rosdep to install missing dependencies. Then I used colcon build --symlink-install because this is a ROS 2 workspace and I sourced install/setup.bash because that makes the built ProjectN packages visible to ros2.

9. I verified that the packages existed

After building, I checked:

```
ros2 pkg list | grep projectn
```

The expected result was:

```
projectn
projectn_interfaces
```

Then I checked the available executables:

```
ros2 pkg executables projectn
```

The expected result included things like:

```
projectn server
projectn agent
projectn projectn-launch-random-tree
projectn agent_hub_robot
projectn robot_node
```

This confirmed that the package was built and sourced correctly. If projectn is not found, it usually means either the workspace was not built or install/setup.bash was not sourced.

10. I added automatic environment setup

To avoid sourcing everything manually every time, I added these lines to ~/.bashrc:

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
echo "source ~/projectn/ROS2Coordination/communication_infra/install/setup.bash" >> ~/.bashrc
echo "export ROS_DOMAIN_ID=77" >> ~/.bashrc
source ~/.bashrc
```

I did this so that every new Ubuntu terminal automatically loads ROS 2, the communication infrastucture workspace, and the same ROS domain ID. The ROS_DOMAIN_ID=77 is used so that all nodes communicate in the same ROS 2 domain.

11. I ran the communication system using three terminals

The system is run using three separate Ubuntu terminals: Tree, Agent, and Robot. Each terminal needs to be in the workspace:

```
cd ~/projectn/ROS2Coordination/communication_infra
```

If .bashrc is configured, I do not need to source manually every time. If not, I manually run:

```
source /opt/ros/humble/setup.bash
source install/setup.bash
export ROS_DOMAIN_ID=77
```

Terminal 1 - I started the communication tree

```
cd ~/projectn/ROS2Coordination/communication_infra
ros2 run projectn projectn-launch-random-tree -- --root Root --nodes 6
```

This starts the tree structure used by the communication infrastructure. I left this terminal running.

Terminal 2 - I started the agent

```
cd ~/projectn/ROS2Coordination/communication_infra
ros2 run projectn agent_hub_robot --agent A1
```

The agent showed a list of available tree nodes. I selected one by typing its number, for example 1. Then the agent stayed running. The agent is the bridge between the robot and the communication tree.

Terminal 3 - I started the robot

In the third terminal, I first prepared the robot command file:

```
cd ~/projectn/ROS2Coordination/communication_infra
mkdir -p ~/.projectn
cp src/projectn/projectn/tools/config/robot_commands.yaml ~/.projectn/robot_commands.yaml
```

Then I exported the command file path:

```
export START_COMMANDS_FILE=~/.projectn/robot_commands.yaml
export ROS_SETUP=/opt/ros/humble/setup.bash
```

Then I started the robot node:

```
ros2 run projectn robot_node --robot R1
```

When it asked whether I wanted to create an agent for this robot, I typed n because I had already started the agent manually in Terminal 2. Then, inside the robot prompt, I attached the robot to the agent and tested commands:

```
attach A1
commands show
send echo hello
```

Important: I did not type robot> myself. If the terminal shows robot>, I only type the command after it, for example attach A1.

12. The normal daily run after setup

Terminal 1

```
cd ~/projectn/ROS2Coordination/communication_infra
ros2 run projectn projectn-launch-random-tree -- --root Root --nodes 6
```

Terminal 2

```
cd ~/projectn/ROS2Coordination/communication_infra
ros2 run projectn agent_hub_robot --agent A1
```

Then I choose a tree node from the menu.

Terminal 3

```
cd ~/projectn/ROS2Coordination/communication_infra
export START_COMMANDS_FILE=~/.projectn/robot_commands.yaml
export ROS_SETUP=/opt/ros/humble/setup.bash
ros2 run projectn robot_node --robot R1
```

Then inside the robot:

```
n
attach A1
commands show
send echo hello
```

13. Common problems I ran into

Problem: Package 'projectn' not found

This means the project was not sourced or not built. Fix:

```
cd ~/projectn/ROS2Coordination/communication_infra
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 pkg list | grep projectn
```

If still not found, rebuild:

```
colcon build --symlink-install
source install/setup.bash
```

Problem: ros2: command not found

```
source /opt/ros/humble/setup.bash
```

Problem: source install/setup.bash: No such file or directory

```
cd ~/projectn/ROS2Coordination/communication_infra
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash
```

Problem: copying the ZIP fails

```
mkdir -p ~/projectn
ls /mnt/c/Users
```

Then I use the correct Windows username in the copy path.

Problem: WSL2 virtualization error

```
wsl.exe --install --no-distribution
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
bcdedit /set hypervisorlaunchtype Auto
```

Then I restart Windows.

14. Rebuilding after changes

If code changes are made, I rebuild from the communication workspace:

```
cd ~/projectn/ROS2Coordination/communication_infra
colcon build --symlink-install
source install/setup.bash
```

If there are strange build issues, I clean and rebuild:

```
cd ~/projectn/ROS2Coordination/communication_infra
rm -rf build install log
colcon build --symlink-install
source install/setup.bash
```

Summary

- I enabled virtualization in BIOS.
- I installed and enabled WSL2.
- I installed Ubuntu 22.04.
- I installed ROS 2 Humble inside Ubuntu.
- I copied the project into the Ubuntu filesystem.
- I built the workspace from ~/projectn/ROS2Coordination/communication_infra.
- I sourced /opt/ros/humble/setup.bash and install/setup.bash.
- I ran the system with three terminals: Tree, Agent, and Robot.

The most important point is that the project must be built and run from:

```
~/projectn/ROS2Coordination/communication_infra
```

because that is the ROS 2 workspace for the communication infrastructure.